

AI IN ENGINEERING

Hackathon

How you solve it counts as much as what you build.

16 – 23 September 2026 | System Modelling | Teams of 4

The method counts as much as the outcome

We are not only asking what you built. We are asking how you got there — how you framed the problem, who you talked to, what you tried and threw away.

55

points for how you worked

45

points for what you built



Using AI is expected

Claude, ChatGPT, Copilot — whatever works.
This is an AI-in-engineering hackathon.

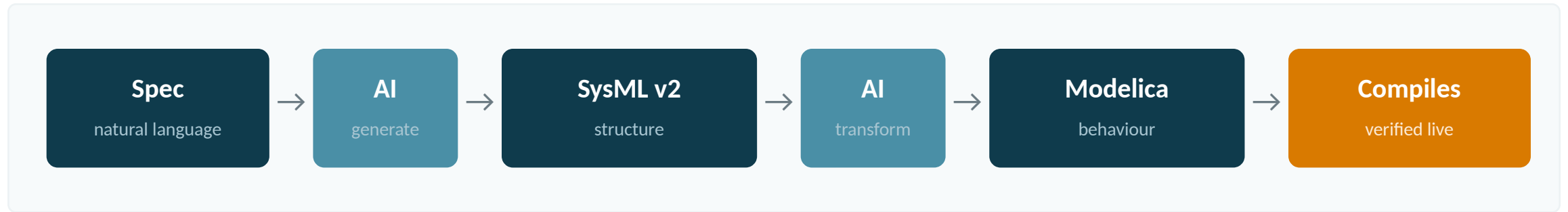
What we score is your judgment in using it:
where you delegated, where you overrode the
output, where you threw it away.

What is not acceptable is passing AI-generated process history off as your own reasoning.

Both weigh the same because the next problem will be different. What transfers is how you work, not what you shipped this week.

One domain: System Modelling

A domain we are deliberately building capability in. Nobody starts fluent — that is the point, and the scoring is calibrated for it.



The Modelica must compile. Binary condition, checked live on a judge-supplied spec. No amount of explanation substitutes for it.

Set your toolchain up on day one. Tool friction, not modelling, is what sinks teams in this domain.

Tooling

SysML v2 — open-source toolchain

Modelica — OpenModelica, verified with omc
MSL available and usable

TIMELINE

Eight days, part-time

Wed 16 Sep	Announced. Work begins on provided problem statements.
Thu 17 Sep, EOD	Self-proposed ideas confirmed or rejected.
Fri 18 Sep	Participation freezes.
Wed 23 Sep, 23:59	Hard stop. Last commit. Repos snapshotted.
Thu 24 - Fri 25 Sep	Live evaluation, 45 minutes per team.
Fri 25 Sep	Results and prizes.

12–15

focused hours per person,
alongside regular work

Scope accordingly

A narrow subsystem that generates,
compiles and holds up under
questioning beats an ambitious plant
model that does none of those.

Four things. Nothing else is read.

1

Git repository

Created on day one. Everything lives here.

2

DECISIONS.md

One commit a day. Three lines. Two minutes.

3

AI-LOG.md

Where you used AI — and what you rejected.

4

8 slides, max

Plus a live demo at the evaluation.

Your repo must contain at least one generated Modelica model that compiles, together with the command to compile it. Judges run that command.

No written report. We would only be reading something a model wrote at 11pm on the 23rd.

DECISIONS.md — the daily log

One commit every day from your start date through 23 September, weekend included.

```
D3 | Dropped rule-based SysML emitter for LLM generation
    | Emitter failed on 4/10 sample specs
D4 | Scoped to a single thermal subsystem
    | Full plant model unreachable at part-time hours
D5 | Added compile-check loop before returning model
    | 6 of 10 first-pass outputs failed omc
```

Format: D<n> | what we decided | why the alternative lost

Two minutes a day. It is the difference between having a process and telling a story about one.

During your evaluation, judges pick three of these entries — their choice — and ask what the alternative was and why it lost.

Timestamps are the evidence

Retro-added entries show in the git log and carry no weight.

A missed day is a missed day

There is no making it up.

Log what you killed

Discarded options are the entries judges pick to question.

Show us where the AI was wrong

Minimum two documented cases where you overrode, corrected or discarded AI output — what you did instead, and why. Both are examined live.

Scores well

- Three approaches generated, two killed for stated reasons
- Plausible-looking Modelica that would not compile
- Invented library components you caught and replaced
- SysML that parsed but did not mean anything

Scores near zero

- “We asked Claude, it worked.”
- A log with no rejections in it
- Hiding AI use entirely
- Overrides you cannot explain when asked

The most valuable entries are the ones where the model was confidently wrong. Those are the ones judges will pick.

How the 100 points split

45

Method

Empathize, Define, Ideate,
Prototype discipline, Test & Iterate

10

AI Collaboration

Delegation judgment
Rejection quality

45

Outcome

Functional depth, engineering soundness,
domain correctness, scope judgment

+5 stretch bonus

Validated simulation results, a traceable SysML → Modelica transformation, verification or parameter sweeps, round-trip consistency checks.

One gate

If the individual probes show only one member can explain the design, Method is capped at 27 of 45 — whatever the artifacts look like.

What earns the 45 method points

Empathize	10	You spoke to a domain expert and something changed — scope cut, problem reframed, assumption killed. No visible delta, no marks.
Define	10	The problem you ended with is sharper than the one you started with, and you can say what sharpened it.
Ideate	8	Real alternatives generated and rejected for stated reasons. One idea carried straight to build scores near zero.
Prototype discipline	7	Incremental commits, working slices early. Evidence you built to learn, not to finish.
Test & Iterate	10	Tested against something real — expert review, compile failures, adversarial specs — and the results changed the build.

Failures documented honestly score higher than a clean story.

The outcome ladder

Anchors, not thresholds — intermediate work scores between them. Everything is demonstrated live on an input the judges supply.

Entry

~10/45

Valid SysML v2 for a narrow subsystem. Parses and validates. No working Modelica.

Floor

~20/45

Plus AI-generated Modelica for the same system. Compiles under omc on a prepared case.

Solid

~32/45

Compiles on a judge-supplied input. SysML covers 3+ element types. Bad input fails gracefully.

Full

45/45

Modelica derived from the SysML, correspondence shown. Compiles and simulates. Defensible under expert review.

Compiling is not the same as being right. A model can compile cleanly and be physical nonsense — domain correctness is assessed separately, by expert review.

45 minutes, fixed structure

Published in advance so you prepare for the right thing.

5

min

Judges read

Silent review of your repo and commit sheet. You set up the demo. No presenting.

10

min

Demo & compile check

Live, uninterrupted. Generate from a judge-supplied spec and run the compiler.

10

min

Decision walk

Judges pick three entries from your DECISIONS.md — their choice — and ask what lost and why.

10

min

Individual probes

Each of the four answers on a part of the work they did not build.

10

min

AI collaboration

Walk through your two override cases.

Bring your toolchain working. Setup comes out of your demo slot, and a compiler that will not start is a compile failure.

Your repo is read cold, in five minutes. A sprawling log is worse than a short one.

Things worth knowing before you start

1

Evidence beats narrative

Every repo is snapshotted at the hard stop and a commit sheet generated — commits per day, lines changed, whether the log was actually committed. A process story that does not match the trail scores zero on Method. Not partial credit.

2

Judges may be your experts

Unavoidable at our size, so it is handled mechanically: each judge declares which teams interviewed them, and their Empathize score for those teams is dropped. Interviewing a judge confers no advantage.

3

Self-proposed problems

Accepted or rejected as submitted — no negotiation, no resubmission. Must capture complexity, fit the domain, have a reachable in-house stakeholder, and be completable at part-time scale.

The honest version of a messy week scores better than a clean invented one. In a domain none of us has worked in before, there will be plenty of mess.

WHAT YOU ARE COMPETING FOR

Prizes

First

Second

Third

On total score

Best Process

Highest Method score in the field.

Boldest Exploration

The team that travelled furthest from where it started.

Decided independently of ranking

Before you start

- 1 Set up your toolchain on day one — not day four
- 2 Commit DECISIONS.md every single day
- 3 Talk to an expert early, and let it change something
- 4 Kill your worse ideas in writing
- 5 Scope to a subsystem you can actually finish
- 6 Make it compile
- 7 Show us where the AI was wrong

Full evaluation criteria in the published methodology document.